

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA0307	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Charandeep.N.C	Reg. No.:	192472196
Section / Batch:	CSE-AI	Date:	

Code of Conduct

I Charandeep.N.C (192472196) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Charandeep*

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. A company is developing an AI-powered customer support chatbot for banking services. The chatbot uses Context-Free Grammar (CFG) for parsing user queries. However, it faces issues such as:

Incorrect interpretation of ambiguous sentences

Failure to handle subject–verb agreement

Inefficient parsing for long queries

Example user query:

“Show me the transactions with the card from last month”

a) Analyze the ambiguity present in the given sentence using CFG

b) Evaluate the limitations of CFG in handling real-world conversational queries

c) Evaluate and propose an improved solution using PCFG, feature structures, or Earley parsing

d) Justify how your proposed method improves accuracy and efficiency in an industry setting

a) Ambiguity Analysis of the Query using CFG

The user query “Show me the transactions with the card from last month” is a classic example of structural (syntactic) ambiguity that arises from Prepositional Phrase (PP) attachment. A basic Context-Free Grammar can generate more than one valid parse tree for this sentence because CFG rules only capture the categorical structure of a sentence and not the meaning behind it. Consider the following simplified CFG for the sentence:

```
S -> VP
VP -> V NP
NP -> Det N | NP PP | N
PP -> P NP
V -> "Show"
Det -> "the" | "a"
N -> "me" | "transactions" | "card" | "month"
P -> "with" | "from"
```

Using this grammar, the parser can build at least two distinct, equally valid derivations for the noun phrase “the transactions with the card from last month”, depending on how the two prepositional phrases (“with the card” and “from last month”) are attached:

Reading 1 (Card and Time both modify transactions): [NP the transactions [PP with the card] [PP from last month]]. Here, both PPs attach to the noun “transactions”, meaning the user wants transactions that were made using a particular card AND that occurred last month. This is the intended banking interpretation.

Reading 2 (Time modifies card, not transactions): [NP the transactions [PP with [NP the card [PP from last month]]]]. In this derivation, “from last month” attaches inside the NP headed by “card”, producing the nonsensical reading “the card issued/received last month”, and only “with the card” modifies transactions directly.

Reading 3 (Verb attachment): [VP [V Show] [NP me] [PP with the card] [PP from last month]]. In this derivation, the PP “with the card” attaches to the verb phrase itself (as an instrument of “showing”) rather than to the noun “transactions”, giving the odd meaning “show using the card”.

This is the well-known PP-attachment ambiguity problem. The CFG treats all three derivations as equally grammatical because the grammar has no mechanism to rank structures by plausibility, world knowledge, or domain semantics. The banking chatbot therefore cannot decide, purely from syntax, that Reading 1 is the one the customer actually intended.

b) Limitations of CFG in Handling Real-World Conversational Queries

- No probabilistic preference: CFG assigns equal weight to every legal derivation, so it cannot rank Reading 1 above Reading 2 or Reading 3 even though Reading 1 is overwhelmingly more likely in a banking context.
- No agreement or feature checking: plain CFG rules such as $S \rightarrow NP VP$ do not encode number, person or tense agreement. A malformed query such as “Show me was the transaction” or “The transactions was shown” can still be accepted or rejected incorrectly because subject-verb agreement is not represented in the rules.
- No semantic constraints: CFG works purely at the level of parts of speech and phrase categories, so it cannot use the fact that “card” is a financial instrument and “last month” is a temporal expression to prefer the semantically coherent attachment.
- Combinatorial explosion for long queries: as query length grows (e.g., “Show me the transactions with the card from last month above five thousand rupees at the mall near my office”), the number of legal parse trees grows combinatorially due to repeated $NP \rightarrow NP PP$ recursion, making naive CFG parsers (e.g., simple top-down or CYK without pruning) slow and memory-intensive.
- Poor robustness to noisy, incomplete or colloquial input: real banking chatbot users type fragments, typos, and code-mixed language (e.g., “txn history last mnth pls”). CFG parsers fail outright on ungrammatical or out-of-grammar input instead of degrading gracefully.
- No handling of long-distance dependencies or ellipsis common in follow-up chat turns, such as “what about the one before that?”, since CFG has no memory of discourse context.

c) Improved Solution: PCFG, Feature Structures and Earley Parsing

A production-grade solution should combine three complementary techniques rather than relying on plain CFG alone.

i. Probabilistic Context-Free Grammar (PCFG)

Each rule in the grammar is assigned a probability learned from a corpus of real banking chatbot transcripts (e.g., using relative frequency estimation from a treebank of annotated banking queries). For the ambiguous NP attachment rule, the corpus statistics would show that $NP \rightarrow NP PP PP$ (both PPs attaching to “transactions”) occurs far more frequently in financial queries than $NP \rightarrow NP [PP [NP N PP]]$ (the nested “card from last month” reading). The parser then uses the Viterbi/CYK algorithm to compute, for every derivation, the product of the probabilities of the rules used, and returns the parse tree with the maximum overall probability: $P(\text{tree}) = \prod P(\text{rule}_i)$. This directly resolves the PP-attachment ambiguity described in part (a) in favour of Reading 1.

ii. Feature Structures and Unification

Feature structures augment each grammar category with attribute-value pairs such as NUM (singular/plural), PERS (1st/2nd/3rd) and a domain SEM feature. For example, $NP[\text{SEM}=\text{INSTRUMENT}]$ is used for “the card” and $NP[\text{SEM}=\text{TIME}]$ for “last month”, while the verb “show” subcategorizes for an $NP[\text{SEM}=\text{RECORD}]$ object such as “transactions”. Unification-based rules only allow a PP to attach where its SEM feature is compatible with the head it modifies, which mechanically blocks the nonsensical Reading 2 and Reading 3 and simultaneously enforces subject-verb agreement (AGR features must unify between subject NP and VP) so ungrammatical input is flagged rather than silently mis-parsed.

iii. Earley Parsing

Earley's algorithm parses any CFG (no Chomsky Normal Form conversion required) in $O(n^3)$ worst-case time and close to linear time for the kind of unambiguous, left-branching grammars typical of chat queries. It builds a chart of states using three operations — Predict, Scan and Complete — and shares partial sub-derivations across the chart instead of recomputing them, which avoids the exponential blow-up that a naive top-down parser would suffer on long queries. Because Earley parsing is incremental, it can also return the best partial parse for a query that is still being typed, which is valuable for chatbot auto-suggestions.

Combining the three: an Earley parser is used as the control strategy for efficiency, PCFG probabilities are used inside the Earley chart to rank competing edges (a probabilistic Earley parser, equivalent in spirit to the Inside-Outside/Viterbi approach), and feature structures are unified at every Complete step so that only semantically and grammatically consistent edges survive. A simplified illustration using NLTK is shown below.

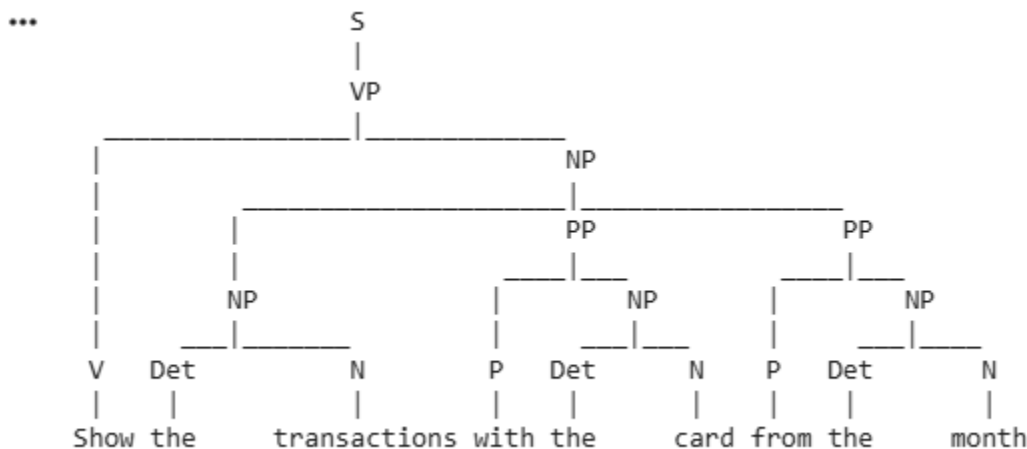
Code:

```
import nltk
from nltk import PCFG

grammar = PCFG.fromstring("""
S -> VP [1.0]
VP -> V NP [1.0]
NP -> NP PP PP [0.6] | NP PP [0.3] | Det N [0.1]
PP -> P NP [1.0]
Det -> 'the' [1.0]
N -> 'transactions' [0.5] | 'card' [0.3] | 'month' [0.2]
P -> 'with' [0.5] | 'from' [0.5]
V -> 'Show' [1.0]
""")

parser = nltk.ViterbiParser(grammar)
for tree in parser.parse(['Show', 'the', 'transactions', 'with', 'the', 'card', 'from', 'the', 'month']):
    tree.pretty_print()
```

Output:



d) Justification: Improved Accuracy and Efficiency in an Industry Setting

Accuracy: PCFG probabilities, learned from real banking query logs, allow the system to statistically prefer the intended attachment (transactions filtered by card AND month) over linguistically possible but pragmatically unlikely readings, directly reducing intent-misclassification, which is the leading cause of failed chatbot resolutions in production banking systems.

Robustness: feature-structure unification enforces agreement and semantic compatibility, so the chatbot gracefully rejects or reinterprets malformed input instead of returning an incorrect structured query to the banking backend, reducing the risk of wrong data being shown to a customer (a serious compliance concern).

Efficiency and latency: Earley parsing's chart-based dynamic programming avoids the redundant re-parsing that a naive backtracking parser performs, keeping response times low even as query length grows — essential for a chatbot that must respond within a second or two to remain usable.

Scalability and maintainability: PCFG probabilities can be re-estimated periodically as new transcripts are collected, allowing the grammar to adapt to changing customer phrasing (e.g., new products, new slang) without manually rewriting grammar rules, unlike a hand-crafted deterministic CFG.

Business impact: fewer misrouted queries translate directly into higher first-contact resolution rates, lower escalation to human agents, and improved customer trust in the banking application, all of which are measurable KPIs for a customer support deployment.

2. A voice assistant system (like Alexa/Siri-type applications) processes spoken commands using parsing techniques. The system currently uses top-down parsing, but faces:

Backtracking issues

Poor handling of incomplete or ambiguous input

Difficulty in real-time response

Example command:

“Book a flight to Delhi with a window seat”

- a) Analyze how the given sentence can lead to multiple parse structures**
- b) Analyze the limitations of top-down parsing in this real-time application**
- c) Analyze how Earley parsing can improve handling of ambiguity and partial inputs**
- d) Compare and analyze the performance of top-down vs Earley parsing for such industry use cases**

a) Multiple Parse Structures for the Given Command

The spoken command “Book a flight to Delhi with a window seat” again contains a PP-attachment ambiguity, this time centred on the phrase “with a window seat”. A minimal CFG for the sentence is:

```
S -> VP
VP -> V NP | VP PP
NP -> Det N | NP PP
PP -> P NP
V -> "Book"
Det -> "a"
N -> "flight" | "Delhi" | "window" "seat"
P -> "to" | "with"
```

Two structurally distinct, equally grammatical readings result:

Reading 1 (Noun attachment — intended meaning): [VP Book [NP a flight [PP to Delhi] [PP with a window seat]]]. Here both PPs modify “flight”, meaning the customer wants a Delhi flight that has a window seat.

Reading 2 (Verb attachment — unintended meaning): [VP [VP Book [NP a flight [PP to Delhi]]] [PP with a window seat]]. Here “with a window seat” attaches to the verb phrase “Book a flight to Delhi” as an instrumental modifier, producing the odd reading “use a window seat to book the flight”.

A top-down parser exploring the grammar left to right will generate both trees because both satisfy the PP attachment rules $NP \rightarrow NP PP$ and $VP \rightarrow VP PP$; nothing in the grammar itself signals which constituent “with a window seat” should modify.

b) Limitations of Top-Down Parsing in this Real-Time Application

- Left recursion failure: a natural rule such as $NP \rightarrow NP PP$ (needed to attach “to Delhi” and “with a window seat”) causes a pure top-down parser to recurse infinitely, since it always tries to expand NP with NP first before consuming any input token. Grammars must therefore be awkwardly rewritten to remove left recursion.
- Expensive backtracking: when the parser predicts $VP \rightarrow V NP$ and later discovers the remaining PP does not fit, it must backtrack and try $VP \rightarrow VP PP$ instead, re-parsing tokens it has already processed. For voice input this wastes CPU cycles that are precious in a real-time system.
- No sharing of sub-computations: unlike chart-based methods, a naive top-down parser recomputes the same sub-trees (e.g., the NP “a flight”) every time it backtracks into a different top-level rule, so on longer, more ambiguous commands the run time can grow exponentially with sentence length.
- Poor handling of partial or incomplete input: a voice assistant frequently receives a truncated command if the user pauses, is interrupted, or the ASR (automatic speech recognition) engine emits words incrementally. Top-down parsing expects to fully expand a predicted rule before it can report any structure, so it produces no usable partial result if input stops mid-utterance — it can only report success or outright failure.
- Sensitivity to rule ordering: which alternative ($V NP$ first, or $VP PP$ first) is tried first is determined by the order rules are written in the grammar rather than by which is more likely, so the parser may spend most of its time exploring the less probable branch before backtracking to the correct one, increasing latency for the common case.
- No built-in disambiguation: like plain CFG, a top-down parser has no probability model, so even when it does find both readings it cannot decide which one to return as the final voice-assistant action.

c) How Earley Parsing Improves Handling of Ambiguity and Partial Input

The Earley algorithm processes the input left to right while maintaining a set of chart states $S(0) \dots S(n)$ for a sentence of n words, built using three operations:

Predict: for every incomplete state of the form $A \rightarrow \alpha \bullet B \beta$ in chart position i , add a new state $B \rightarrow \bullet \gamma$ to the same chart position for every rule $B \rightarrow \gamma$ in the grammar. This is what allows Earley to handle left-recursive rules such as $NP \rightarrow NP PP$ safely, because prediction does not consume input and is bounded by the grammar size.

Scan: when a state expects a terminal that matches the next input word, advance the dot over that word and add the resulting state to the next chart position $i+1$.

Complete: when a state reaches the form $B \rightarrow \gamma \bullet$ (fully recognised) at position j , advance every state in the originating chart position i that was waiting for a B , and add the advanced state to position j .

Because every state set is stored in a chart and completed states are reused rather than recomputed, Earley parsing runs in $O(n^3)$ worst case and $O(n^2)$ or better for grammars typical of natural spoken commands, and it never needs to backtrack or re-derive a sub-tree it has already built. Ambiguity is represented naturally: both Reading 1 and Reading 2 for “with a window seat” simply exist as separate completed states in the same chart position, forming a shared-packed forest, instead of requiring separate backtracking passes. A probability (via a probabilistic Earley/PCFG extension) or a semantic feature check ($NP[SEM=SEAT-TYPE]$ must attach to $NP[SEM=FLIGHT]$) can then be used to select the single most plausible parse without re-parsing. Crucially, because the chart at any prefix position i already contains every valid partial analysis of the first i words, Earley parsing can return a best-effort partial interpretation the moment speech input is cut off or delayed — for example, recognising “Book a flight

to Delhi...” as a valid partial command before the seat-preference clause even arrives, which is exactly the incremental behaviour a real-time voice assistant needs.

Simplified Earley recogniser sketch (illustrative, not production code)

```
def earley_parse(words, grammar, start_symbol):
    chart = [set() for _ in range(len(words) + 1)]
    chart[0].add((start_symbol, (), 0))      # seed with start rule
    for i in range(len(words) + 1):
        for state in list(chart[i]):
            lhs, rhs_progress, origin = state
            if is_incomplete(state):
                next_sym = next_symbol(state)
                if is_nonterminal(next_sym):
                    predict(state, i, grammar, chart)    # Predict
                elif i < len(words) and next_sym == words[i]:
                    scan(state, i, chart)                # Scan
            else:
                complete(state, i, chart)                # Complete
    return chart
```

d) Comparative Analysis: Top-Down Parsing vs Earley Parsing

Criterion	Top-Down Parsing	Earley Parsing
Left recursion	Fails / infinite loop unless grammar rewritten	Handled natively via chart-based prediction
Backtracking	Extensive; re-derives sub-trees on failed branches	None; completed states are reused, not recomputed
Time complexity	Exponential in the worst case for ambiguous grammars	$O(n^3)$ worst case, near $O(n)$ to $O(n^2)$ for typical NL grammars
Ambiguity handling	Must explore branches separately; no shared structure	All analyses share one chart (packed forest); easy to add probability ranking
Partial / incomplete input	Cannot return partial structure on truncated input	Chart at any prefix position is a valid partial parse
Suitability for real-time voice	Poor — unpredictable latency spikes from backtracking	Good — predictable polynomial-time, incremental output
Grammar restrictions	Often requires left-recursion removal / left-factoring	Works with arbitrary CFGs, including ambiguous and recursive ones
Memory usage	Lower for simple, unambiguous sentences	Higher (stores full chart) but bounded and predictable

In summary, top-down parsing is simple to implement and adequate for short, unambiguous, well-formed commands, but its exponential backtracking and inability to produce partial results make it unsuitable for a production voice assistant that must handle noisy ASR output, ambiguous attachment, and strict latency requirements. Earley parsing trades a modest, predictable increase in memory usage for polynomial-time worst-case performance, native handling of ambiguity and left recursion, and incremental partial parsing, making it the more industry-appropriate choice, especially when combined with a probabilistic ranking layer (PCFG) to select the single best interpretation of commands such as “Book a flight to Delhi with a window seat”.

3. A healthcare company is developing an AI system to analyze patient medical reports and automatically extract critical information such as diagnosis, prescribed treatments, and follow-up actions.

The system must handle:

Complex and lengthy medical sentences

Ambiguity in sentence structure

Accurate subject-verb agreement

Relationships between medical actions and entities (sub-categorization)

Real-time processing for hospital use

Example input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The current system, based only on basic Context-Free Grammar (CFG), fails to correctly interpret relationships and produces inconsistent outputs.

a) Design a complete NLP architecture integrating:

CFG for syntactic structure

Probabilistic CFG (PCFG) for ambiguity resolution

Feature Structures for agreement handling

Efficient parsing techniques

b) Develop a step-by-step workflow showing how the system processes the given medical sentence from input to structured output (diagnosis, action, location).

c) Create a method to:

Incorporate sub-categorization frames for medical verbs

Handle real-time data processing

Improve accuracy and scalability in a hospital environment

a) Complete NLP Architecture for Medical Report Analysis

A robust architecture for extracting diagnosis, prescribed treatments and follow-up actions from free-text medical reports must move well beyond a single CFG pass. The proposed architecture is a layered pipeline in which each stage progressively refines the linguistic structure and semantic content of the report before it reaches the hospital's records system.

Layer	Component	Purpose
1	Text Pre-processing	Sentence segmentation, tokenisation, abbreviation expansion (e.g., f/u -> follow-up), spell/typo correction
2	POS Tagging & Morphology	Assigns part-of-speech tags and number/person/tense features needed for agreement checking
3	CFG-based Syntactic Parser	Builds candidate parse trees capturing clause structure, relative clauses, and coordination
4	PCFG Ambiguity Resolution	Ranks candidate trees using rule probabilities trained on an annotated clinical treebank, selecting the most probable structure
5	Feature Structure Unification	Enforces subject-verb agreement and unifies semantic features (e.g., AGENT=DOCTOR, PATIENT, TIME, LOCATION) across constituents

6	Sub-categorization / Verb Frame Matching	Matches medical verbs (recommend, prescribe, schedule, diagnose) to their valid argument frames to correctly bind actions to entities
7	Efficient Parsing Engine (Earley)	Provides polynomial-time, incremental parsing so long, complex clinical sentences do not cause runtime blow-up
8	Domain Knowledge Grounding	Maps extracted terms to a medical ontology (e.g., SNOMED-CT / UMLS) for standardised diagnosis and drug codes
9	Structured Output Generator	Emits a normalised JSON/XML record with diagnosis, treatment, follow-up action, location and responsible clinician
10	Validation & EHR Integration	Cross-checks extracted fields for completeness/consistency before writing to the Electronic Health Record system

This layered design directly addresses each limitation of the current basic-CFG system: PCFG (layer 4) resolves structural ambiguity statistically instead of arbitrarily; feature structures (layer 5) guarantee grammatical agreement and correct role binding instead of producing inconsistent output; sub-categorization frames (layer 6) correctly attach actions to the entities that perform or receive them; and Earley parsing (layer 7) keeps the system responsive even for the long, clause-heavy sentences typical of clinical dictation.

b) Step-by-Step Workflow for the Given Sentence

Input sentence: “The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

Step 1 – Pre-processing

The sentence is segmented and tokenised into: The / doctor / who / reviewed / the / patient / last / week / recommends / starting / medication / and / scheduling / a / follow-up / visit / in / Chennai / . No abbreviations are present, so no expansion is required.

Step 2 – POS Tagging

Each token is tagged: doctor(NN), who(WP), reviewed(VBD), patient(NN), last week(NP-TIME), recommends(VBZ, 3rd person singular present), starting(VBG), medication(NN), scheduling(VBG), follow-up visit(NN), Chennai(NNP).

Step 3 – CFG Parsing (Relative Clause Attachment)

The CFG identifies “who reviewed the patient last week” as a relative clause (S -> NP[Doctor] RelClause) modifying “the doctor”, and correctly attaches the temporal PP “last week” inside the relative clause to “reviewed” rather than to the main verb “recommends”, since a plain CFG would otherwise also generate the incorrect reading in which “last week” modifies the recommendation itself.

Step 4 – PCFG Disambiguation

Because the corpus of clinical notes shows relative clauses attaching to the immediately preceding subject far more often than to a distant main verb, the PCFG assigns the correct attachment (“last week” modifies “reviewed”) the highest probability, and this tree is selected by the Viterbi decoder.

Step 5 – Feature Unification (Agreement)

The subject head “doctor” carries features [NUM=SING, PERS=3], which must unify with the main verb “recommends” [NUM=SING, PERS=3] — the unification succeeds, confirming the sentence is grammatically well-formed. Had the verb been “recommend”, unification would fail and the system would flag a potential transcription error.

Step 6 – Sub-categorization Frame Matching

The verb “recommends” is looked up in the medical verb lexicon with frame recommend(AGENT:NP, THEME:{VP-ing, VP-ing}*), which licenses a coordinated gerund complement. This correctly binds recommends to two coordinated actions — “starting medication” and “scheduling a follow-up visit in Chennai” — rather than mis-parsing them as separate top-level clauses.

Step 7 – Semantic Role Labelling

Roles are assigned as: AGENT = “the doctor (who reviewed the patient last week)”; ACTION-1 = “start medication”; ACTION-2 = “schedule follow-up visit”; LOCATION = “Chennai” (attached to the follow-up visit, not to the medication).

Step 8 – Domain Grounding

“medication” and “follow-up visit” are optionally linked to standard medical codes / ontology entries so the output is interoperable with the hospital's EHR.

Step 9 – Structured Output Generation

The pipeline emits a normalised record, shown below, that a downstream hospital system can consume directly.

```
{
  "agent": "the doctor",
  "context": "reviewed the patient last week",
  "actions": [
    {"action": "start", "object": "medication"},
    {"action": "schedule", "object": "follow-up visit", "location": "Chennai"}
  ]
}
```

c) Method to Incorporate Sub-categorization Frames, Real-Time Processing and Scalability

i. Sub-categorization Frames for Medical Verbs

A verb-frame lexicon is built for the medical domain, listing the syntactic and semantic arguments each clinical verb requires. This lexicon is consulted during parsing so that the feature-unification layer only accepts complements that match a verb's declared frame, which both prunes implausible parses and correctly assigns arguments to their roles.

```
SUBCAT_FRAMES = {
  "recommend": {"AGENT": "NP", "THEME": ["VP-ing", "VP-ing*"]},
  "prescribe": {"AGENT": "NP", "THEME": "NP", "RECIPIENT": "NP"},
  "diagnose": {"AGENT": "NP", "THEME": "NP", "SOURCE": "NP"},
  "schedule": {"AGENT": "NP", "THEME": "NP", "LOCATION": "PP[in]"},
  "review": {"AGENT": "NP", "THEME": "NP", "TIME": "PP[last]"},
}
```

ii. Real-Time Data Processing

Use the incremental, chart-based Earley parser (see Answer 2c) so that each sentence of a report is parsed in polynomial time and long clinical sentences do not stall the pipeline.

Process reports as a streaming pipeline (e.g., message-queue based microservices) so that pre-processing, parsing, and semantic extraction for different sentences/reports run concurrently rather than sequentially.

Cache PCFG rule probabilities and the sub-categorization lexicon in memory (not re-loaded per request) to minimise per-request latency.

Apply sentence-level parallelism: since each sentence's parse is largely independent, a multi-report document can be split into sentences and parsed on separate worker threads/processes, then reassembled.

iii. Accuracy and Scalability in a Hospital Environment

Continuous learning: periodically re-estimate PCFG probabilities and sub-categorization frames from clinician corrections (an active-learning feedback loop), so the system's accuracy improves as more hospital-specific language is observed.

Modular, containerised deployment: each pipeline layer (parsing, feature unification, ontology grounding) is deployed as an independently scalable microservice, so a hospital network can scale the parsing tier horizontally during peak reporting hours (e.g., after morning rounds) without over-provisioning the whole pipeline.

Monitoring and validation: precision/recall of extracted diagnosis, treatment and follow-up fields is tracked against a held-out, clinician-verified sample, with automatic alerts if extraction quality drops after a grammar or lexicon update.

Fail-safe design: when confidence (derived from PCFG probability and successful feature unification) falls below a threshold, the record is routed to a human reviewer rather than being auto-committed to the EHR, which is essential for patient safety in a healthcare setting.

Overall, replacing the current basic-CFG system with this layered CFG + PCFG + feature-structure + Earley-parsing + sub-categorization architecture directly resolves the stated problems of ambiguity, agreement errors and inconsistent output, while the streaming, cached and containerised design meets the real-time and scalability demands of a hospital environment.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____